

Technical Report: Practical Byzantine Fault Tolerance (pBFT)

Aldo Luna Bueno
20170325B

July 2, 2026

Contents

1	Foundations and Motivation	2
1.1	From “Fail-Stop” to “Byzantine Fault”	2
1.2	The pBFT System Model	2
1.3	The $3f + 1$ Rule	2
2	The Protocol in the Normal Case	3
2.1	State Machine Replication	3
2.2	The 3-Phase Protocol	3
3	Garbage Collection and Checkpoints	3
3.1	Stable Checkpoint Generation	4
3.2	Watermarks and State Transfer	4
4	View Changes (Leader Fault Tolerance)	4
5	Cryptography, Practical Optimizations, and Performance	5
5.1	Byzantine File System (BFS) Architecture	5
6	Advanced Concepts and Contemporary Use Cases	6
6.1	The pBFT Renaissance	7
7	Conclusions	7

1 Foundations and Motivation

1.1 From “Fail-Stop” to “Byzantine Fault”

In the 1980s, fault tolerance research was heavily concentrated on *fail-stop* faults, which occur when a node simply ceases to operate, typically due to a hardware crash. However, as systems grew more complex, these failures became just the tip of the iceberg.

A node can fail due to software bugs or be compromised by malicious attackers. In these scenarios, the defective node does not shut down; it continues to communicate but can behave arbitrarily or transmit false information. This arbitrary behavior is known as a **Byzantine Fault**. Within a Byzantine environment, it is assumed that defective nodes actively conspire to corrupt the system state.

1.2 The pBFT System Model

The pBFT algorithm assumes a realistic and hostile system model:

- **Asynchrony:** It operates in an asynchronous distributed system, such as the Internet, where messages can be delayed, lost, or delivered out of order.
- **Strong Adversary:** It posits an adversary capable of coordinating compromised nodes. pBFT relies heavily on cryptography (public-key signatures, MACs, cryptographic digests) to secure message integrity.
- **Safety without Synchrony:** It guarantees its safety property (contradictory operations will never be committed) regardless of network delays, relying only on weak synchrony for liveness.

1.3 The $3f + 1$ Rule

To tolerate f Byzantine failures, the minimum number of replicas required is $n = 3f + 1$. This is because the protocol must be able to make progress using responses from $n - f$ replicas (since f nodes could be dead or unresponsive). Within those received responses, f nodes could be malicious liars. For the honest nodes’ votes to strictly outvote the malicious ones, the following inequality must hold: $(n - f) - f > f \implies n > 3f$.

Another perspective to understand this rule is through quorum intersections. Committing an operation requires a quorum of $2f + 1$ identical responses. If two distinct quorums of $2f + 1$ intersect, their mathematical overlap will always contain at least $f + 1$ replicas, inherently guaranteeing at least one honest node in the intersection.

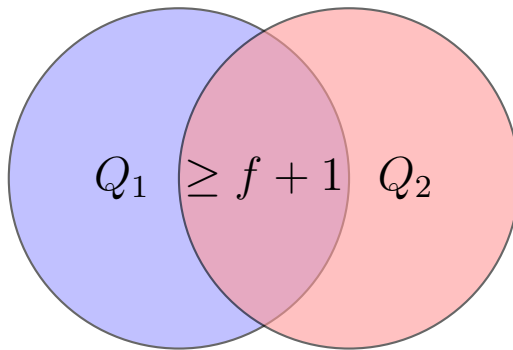


Figure 1: Quorum Intersection in pBFT

2 The Protocol in the Normal Case

2.1 State Machine Replication

pBFT models the service as a deterministic state machine. All replicas must start in the same initial state and execute operations identically. The primary challenge is to guarantee that all honest replicas execute the same requests in the exact same total order. The system progresses through sequences of **views**. In each view, one replica assumes the role of the **primary (leader)**, while the remaining replicas act as **backups**.

2.2 The 3-Phase Protocol

To tolerate Byzantine faults, the protocol requires three phases of communication:

1. **Pre-prepare:** The primary receives a client request, assigns a sequence number to it, and broadcasts a **PRE-PREPARE** message to the backups.
2. **Prepare:** A backup verifies the message and broadcasts a **PREPARE** message to all other replicas. A node considers itself “prepared” once it has collected the original request/message plus a quorum of $2f$ matching **PREPARE** messages.
3. **Commit:** Upon becoming “prepared,” the node broadcasts a **COMMIT** message. When a node aggregates $2f + 1$ matching **COMMIT** messages, it achieves local commit status, executes the operation, and returns the response to the client.

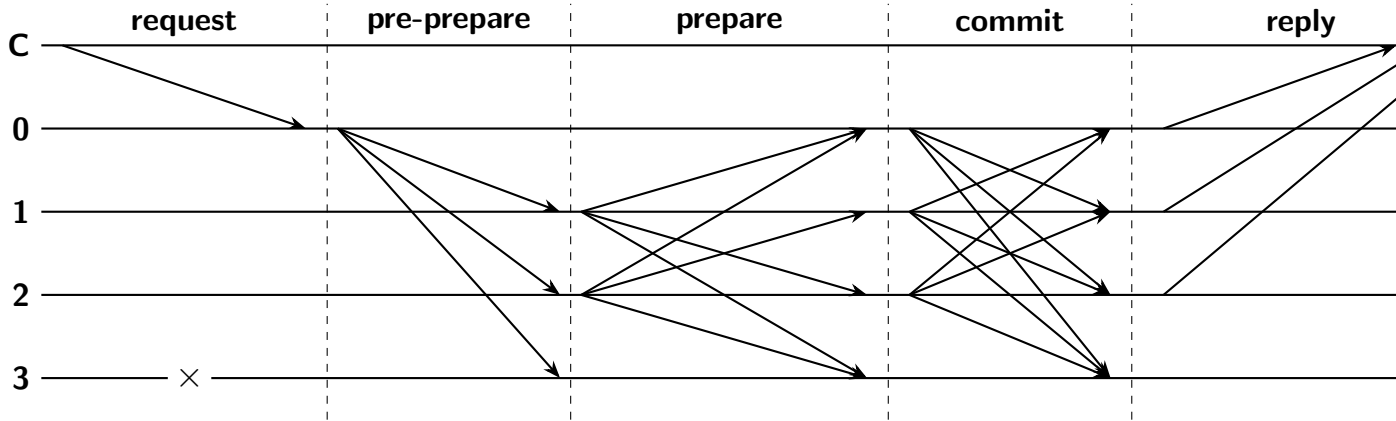


Figure 2: Sequence diagram of the normal case operation where node 3 is faulty

3 Garbage Collection and Checkpoints

In pBFT, each replica must store messages in a log. However, memory resources necessitate a garbage collection mechanism to periodically truncate this log so that it does not grow indefinitely.

3.1 Stable Checkpoint Generation

The protocol periodically generates cryptographic proofs (e.g., every 100 requests) confirming that a consolidated state is correct, which is referred to as a **checkpoint**. When a replica generates a checkpoint, it broadcasts a **CHECKPOINT** message to all other replicas.

This checkpoint transitions into a **stable checkpoint** once the replica successfully aggregates a total of $2f + 1$ identical checkpoint messages from other nodes. This quorum proves that an honest majority of the system agrees on the current state. Upon achieving stability, the replica safely purges all messages related to the **pre-prepare**, **prepare**, and **commit** phases that have sequence numbers equal to or lower than that of the stable checkpoint.

3.2 Watermarks and State Transfer

The protocol establishes strict operating boundaries defined by two watermarks: the low watermark (h), which matches the sequence number of the latest stable checkpoint, and the high watermark ($H = h + k$). This range restricts the sequence numbers of valid operations, preventing a malicious leader from intentionally assigning massive sequence numbers to exhaust memory space. Furthermore, pBFT utilizes a hierarchical state tree combined with a *copy-on-write* memory management technique to perform efficient state transfers and mathematical updates without halting the entire system, thereby accelerating the recovery of slow or lagging nodes.

4 View Changes (Leader Fault Tolerance)

To maintain the system’s liveness when the primary node fails or acts maliciously, nodes transition through a succession of configurations called *views*.

Backup nodes monitor the primary’s behavior using timers (*timeouts*). A backup initiates a timer upon receiving a request; if the timer expires without any progress in processing, the backup assumes the leader is defective and halts the execution of the normal protocol.

The node ceases to accept normal operation messages and broadcasts a **VIEW-CHANGE** message requesting a transition to the next view ($v + 1$). To be valid, this message appends cryptographic proofs of its latest stable checkpoint as well as all operations that had already reached the locally prepared state.

The new leader, designated by the formula $v \bmod N$ (where N is the total number of nodes), takes control after aggregating at least $2f$ of these messages. It then issues a **NEW-VIEW** message containing consolidated proofs to resume the interrupted operations, ensuring that no operation already confirmed to a client is ever lost, and using null operations (*no-ops*) to fill any empty slots in the sequence numbers.

To prevent abuses, the protocol implements forced rotation and exponential timeout backing off in case of repetitive, failed view changes.

5 Cryptography, Practical Optimizations, and Performance

pBFT resolves cryptographic latency bottlenecks by reserving expensive public-key signatures (such as RSA) exclusively for critical operational transitions like view changes. In the normal case, it employs symmetric-key cryptography **Message Authentication Codes (MACs)** for the overwhelming majority of interactions, which perform orders of magnitude faster.

Additionally, it optimizes data transmission overhead by instructing only one designated replica to send the complete response payload back to the client, while the remaining nodes transmit only small cryptographic digests. For high-demand environments, pBFT utilizes request batching within a single consensus sequence and enables a fast path for read-only operations.

5.1 Byzantine File System (BFS) Architecture

To empirically demonstrate the "practical" value of pBFT, the authors developed a replication library upon which they implemented the Byzantine-Fault-tolerant File System (BFS). This filesystem supports the standard NFS V2 network protocol transparently, requiring no modifications to either the client or the original kernel server.

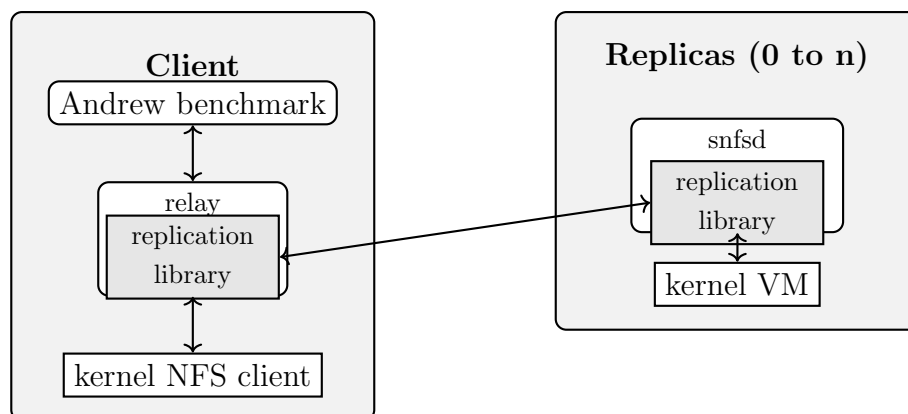


Figure 3: Simplified BFS Architecture

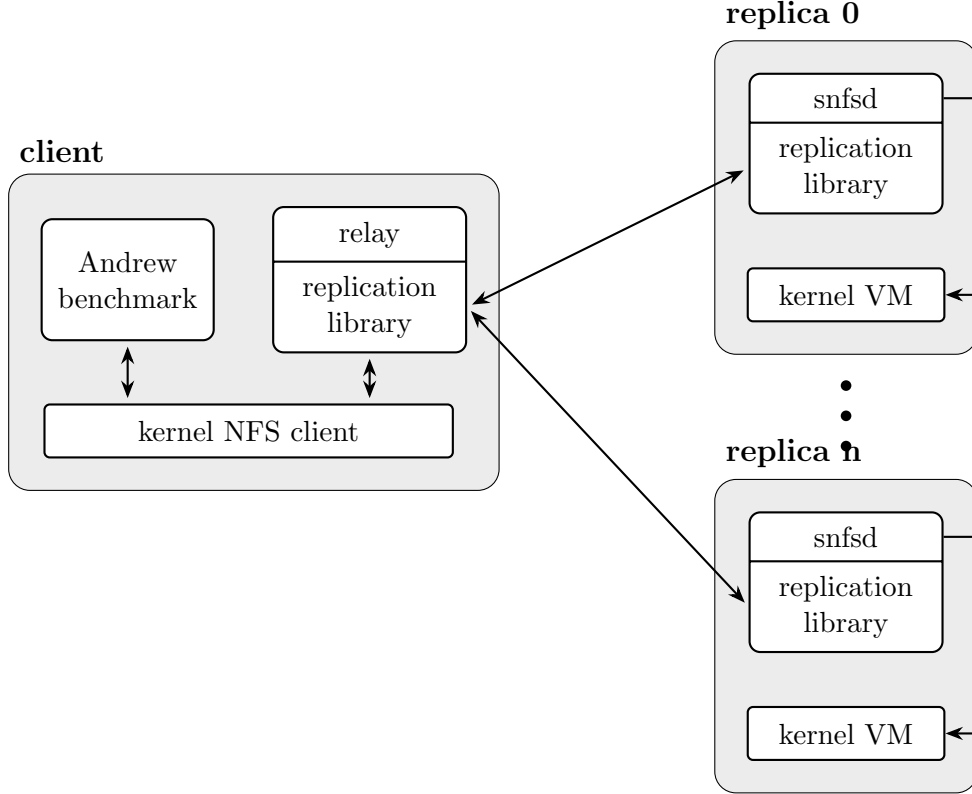


Figure 4: BFS Architecture Diagram

By subjecting BFS to the demanding *Andrew benchmark*, the authors proved that the performance penalty incurred is remarkably small:

Benchmark Phases	BFS (pBFT)	BFS-nr (Unrep.)	NFS-std (Commercial)
1. Create directories	0.55	0.35	1.75
2. Copy files	9.24	5.08	9.46
3. Read metadata	7.24	6.11	5.36
4. Read data	8.77	7.41	6.60
5. Compile and link	38.68	32.12	39.35
Total Time (seconds)	64.48	51.07	62.52

Table 1: Andrew Benchmark Results. The pBFT (BFS) execution time proved to be only roughly 3% slower than or even competitive with industry-standard commercial NFS.

6 Advanced Concepts and Contemporary Use Cases

The base pBFT algorithm assumes that no more than f failures occur during the entire lifespan of the system. However, computing systems operate for months or years, increasing the statistical probability of exceeding this cumulative limit of f failures. To bridge this gap, the system employs a technique called **Proactive Recovery**.

Instead of waiting for failures to be detected, nodes cycle through reboots and recoveries proactively (e.g., every 10–20 minutes) utilizing a secure hardware coprocessor, which stores the original source code in read-only memory (ROM). During this recovery

process, new secret keys are generated to prevent identity theft. This drastically transforms the model: rather than tolerating f failures over its entire lifespan, pBFT tolerates f *simultaneous* failures within a small, defined window of time.

6.1 The pBFT Renaissance

Initially rejected by the industry in favor of prevention models and firewalls, pBFT was long viewed as a purely academic problem. However, it has experienced an explosive renaissance driven by the **Zero Trust** cybersecurity paradigm (assuming the network is already breached) and the innovations brought by the blockchain era.

Unlike the invention of **Bitcoin**, whose Proof-of-Work consensus tolerates malicious participants but heavily penalizes performance (high latency, massive energy consumption, and lack of strict linearizability or immediate finality due to potential *forks*), pBFT offers immediate, absolute finality and high throughput in closed groups. Today, pBFT and its derivatives serve as the architectural standard in financial systems and enterprise *Hyperledgers* such as **Stellar** and **IBM Hyperledger**, solving corporate consensus challenges at commercial speeds.

7 Conclusions

Practical Byzantine Fault Tolerance (pBFT) successfully bridged the gap between theoretical models of Byzantine fault tolerance and real-world distributed systems. By optimizing cryptographic operations—relying primarily on efficient symmetric Message Authentication Codes (MACs) and reserving expensive public-key cryptography only for critical view-change procedures—pBFT demonstrated that resilience against arbitrary, malicious node behavior could be achieved without crippling system performance. The empirical results from the Byzantine-Fault-tolerant File System (BFS) proved that pBFT incurs merely a marginal overhead compared to standard, unreplicated systems.

Furthermore, mechanisms such as garbage collection via stable checkpoints and proactive recovery through secure hardware coprocessors address the limitations of unbounded log growth and long-term security, ensuring the protocol remains robust over extended lifespans.

While initially considered an academic exercise due to the industry’s reliance on perimeter-based security, the rise of the Zero Trust security paradigm and decentralized networks has catalyzed a renaissance for pBFT. Its ability to provide immediate transaction finality, strict linearizability, and high throughput without the massive energy consumption associated with Proof-of-Work mechanisms has established it as the foundational consensus architecture for modern permissioned blockchains, such as Hyperledger, and federated financial networks. Ultimately, pBFT remains a seminal work, proving that achieving high-performance consensus in untrusted environments is both mathematically rigorous and practically viable.